

FTRIA-110 — Runtime Invariant Extension Algebra

Part II — Runtime Invariant Algebra

Sizhe Tan

with chatGPT (OpenAI)

2026-07-15

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

Runtime Invariant Discovery identifies stable structural entities underlying multiple runtime realizations.

Discovery alone, however, is insufficient for intelligent computation.

Once a Runtime Invariant has been identified, it must be extended, transformed, generalized, migrated, reused, and adapted while preserving its structural identity.

This paper introduces **Runtime Invariant Extension Algebra (RIEA)** as the second major branch of Runtime Invariant Algebra.

Rather than viewing extension algorithms as isolated engineering techniques, RIEA interprets them as invariant-preserving operators acting within Runtime Invariant Configuration Spaces.

This perspective unifies numerous existing methods—including CCC Preserve Generation, code refactoring, runtime migration, fine-tuning, model adaptation, and structural generalization—under a common algebraic framework.

1. From Discovery to Extension

Discovery identifies a Runtime Invariant.

Extension determines how that Runtime Invariant evolves.

Conceptually,

Runtime Invariant Discovery

↓

Runtime Invariant

↓

Runtime Invariant Extension



New Runtime Realizations

The Runtime Invariant remains stable,
while its admissible realizations continue to evolve.

2. What Is Runtime Invariant Extension?

Runtime Invariant Extension consists of operations that expand the applicability of a Runtime Invariant without changing its structural identity.

Typical extension objectives include

- generalization,
- refinement,
- reuse,
- migration,
- optimization,
- adaptation,
- composition,
- implementation replacement.

Unlike Discovery,

Extension does not seek a new Runtime Invariant.

Instead,

it preserves an existing Runtime Invariant while enlarging its Configuration Space.

3. Runtime Invariant Extension Operators

Many existing algorithms naturally belong to the Extension family.

Examples include

Structural Intelligence

- CCC Preserve Generation,
- CCC Preserve Coding,
- Dilution,
- Delution,
- Gap Bridging.

Software Engineering

- code refactoring,
- compiler optimization,

- runtime migration,
- implementation replacement.

Modern AI

- Fine-Tuning,
- LoRA,
- PEFT,
- model merging,
- transfer learning.

Although developed independently,
these methods perform a common Runtime Function:
Extend Runtime Invariants while preserving structural identity.

4. Extension Rather Than Replacement

A central principle of RIEA is that intelligent evolution rarely replaces Runtime Invariants.

Instead,
it extends them.

This distinction explains why many successful engineering practices emphasize

- backward compatibility,
- reusable abstractions,
- incremental improvement,
- modular evolution,
- structural preservation.

The Runtime Invariant remains stable,
while the surrounding implementation space expands.

5. Degrees of Freedom Expansion

Extension can be interpreted geometrically.

Discovery identifies a Runtime Invariant.

Extension enlarges the admissible Degrees of Freedom surrounding that Runtime Invariant.

Conceptually,

Before Extension

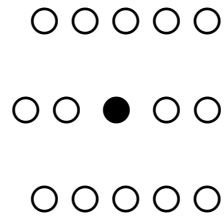
○○○



○○○



After Extension



The Runtime Invariant remains unchanged.
Its admissible Configuration Space becomes larger.

6. Explicit and Implicit Extension

Runtime Invariant Extension may occur explicitly or implicitly.

Explicit Extension

Examples include

- CCC Preserve Generation,
- structural refactoring,
- Runtime migration.

Implicit Extension

Examples include

- Fine-Tuning,
- LoRA,
- PEFT,
- continual learning,
- adaptive optimization.

Different computational mechanisms,
yet the same Runtime Function:
preserving and extending Runtime Invariants.

7. Runtime Invariant Extension Algebra

Runtime Invariant Extension Algebra provides a common language for these operations.

Instead of describing algorithms according to implementation,
RIEA classifies them according to their structural effect on Runtime Invariants.

Examples include

- preserving,

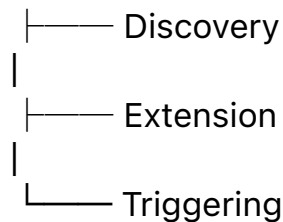
- expanding,
- refining,
- composing,
- projecting,
- migrating,
- adapting.

Each operation transforms Runtime realizations while preserving invariant identity.

8. Relationship to Discovery and Triggering

Runtime Invariant Algebra consists of three complementary branches.

Runtime Invariant Algebra



Discovery identifies Runtime Invariants.

Extension enlarges their admissible Configuration Spaces.

Triggering activates Runtime Invariants during runtime execution.

Together,

they describe the complete lifecycle of Runtime Invariants.

Key Takeaways

- Runtime Invariant Extension begins after Runtime Invariant Discovery.
- Extension preserves Runtime Invariants while expanding their admissible realizations.
- Many existing algorithms—including Preserve Generation, Fine-Tuning, LoRA, PEFT, and runtime migration—share the same Runtime Function.
- Runtime Invariant Extension enlarges Configuration Spaces rather than replacing Runtime Invariants.
- Extension may occur explicitly or implicitly.
- Runtime Invariant Extension Algebra provides a unified interpretation of structural evolution across software engineering and modern AI.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra

- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding
- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery

Extension Algebra

- FTRIA-111 — CCC Preserve Generation as RI-Preserving Transformation
- FTRIA-112 — CCC Preserve Coding as Runtime Invariant Coding
- FTRIA-113 — Dilution as Degrees-of-Freedom Expansion
- FTRIA-114 — Delution as Runtime Invariant Refinement
- FTRIA-115 — Gap Bridging as Runtime Configuration Navigation
- FTRIA-116 — Fine-Tuning, LoRA and PEFT as Runtime Invariant Extension

Trigger Algebra

- FTRIA-120 — Runtime Invariant Trigger Algebra